Experiment 4: Docker Essentials

Topics Covered:

- Dockerfile
- .dockerignore
- Tagging
- Publishing

Part 1: Containerizing Applications with Dockerfile

Step 1: Create a Simple Application

Python Flask App:

mkdir my-flask-app

cd my-flask-app

app.py

```python
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
Flask==2.3.3
```

Step 2: Create Dockerfile

Dockerfile

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

Part 2: Using .dockerignore

Step 1: Create .dockerignore File

.dockerignore

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```

Step 2: Why .dockerignore is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Part 3: Building Docker Images

Step 1: Basic Build Command

```
devops_user@DEVOPS-LAB:/mnt/c/Users/DevOps-Lab/projects/nginx_docker_image$ docker build -t webserver_nginx:latest .
[+] Building 4.3s (14/14) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 332B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [internal] load metadata for docker.io/library/nginx:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:347760018c1ed3113602f256716d36203f33bab1acdada13fcf731bbb
 =>   resolve docker.io/library/python:3.9-slim@sha256:347760018c1ed3113602f256716d36203f33bab1acdada13fcf731bbb
 =>  sha256:c2478d02eb0413a890f06dd422b601a1d01a932ce0ca32ec75af74b366c  2.11kB / 2.11kB
 =>  sha256:1319f5208c0261138607866b7d376b1833cf8d5e014aead6c4a76f392d0  1.28kB / 1.28kB
 =>  sha256:4e3dfa3b32acd14b32982cbab136c5d039b05cfc274da2d8f13439c06ff9a8  ff2a48  29.79MB
 =>  sha256:76c183072263104405ecbabe5205ef7d40212a9aaceb8023dd9ee6c8f29b6d  1.58kB
 =>  extracting sha256:4e3dfa3b32acd14b32982cbab136c5d039b05cfc274da2d8f13439c06ff92a48
 =>  extracting sha256:76c183072263104405ecbabe5205ef7d40212a9aace08023dd9ee6c8f29b6d
 => [2/5] FROM nginx:latest@sha256:6189221721a2ccb1c112c06eeb61e983554541d4ed3fcf2f31bbb
 => resolve nginx:latest  553B
 => [3/5] WORKDIR /app
 => [4/5] COPY requirements.txt .
 => [5/5] RUN pip install --no-cache-dir -r requirements.txt
 => exporting to image
 => => exporting layers
 => => writing image sha256:9001c461a319502a6248a79212cf0c13357c755c7b12321318c1ddfa0042c13b
 => => naming to docker.io/library/webserver_nginx:latest
 => => writing image sha256:907f06d2f41f530a0094f4611813de16a1f1a76aa6c4b27f724adcb50d8b477cd
 => => writing attestation manifest sha256:602ebbb7b165a983f00e6858d1728ededee17b06a4cfee911ed9ef0b1d630b
 => => writing manifest to image destination
 => => unpacking to docker.io/library/webserver_nginx:latest
```

Step 2: Tagging Images

```
PS C:\Users\Anirudh Chawla\OneDrive\Desktop\UPES\6th Sem/Cotainerization And DevOps Lab\experiment-4> docker images
REPOSITORY                      TAG       IMAGE ID       CREATED              SIZE
my-flask-app                    latest    2e6c702628a4   About a minute ago   200MB
web_app_project-backend         latest    a1b113feb6cd   2 days ago           187MB
containerization_web_app-backend latest   eabf88c5f909a  3 days ago           187MB
web_app_project-db              latest    7ec9d7a46981   3 days ago           392MB
containerization_web_app-db     latest    921d26fa4265   3 days ago           392MB
nginx-alpine                    latest    85b49b9478d8   3 weeks ago          16MB
nginx-ubuntu                    latest    db766c1efb20   3 weeks ago          199MB
nginx                           latest    341bf0f3ce6c   6 weeks ago          237MB
<none>                          <none>    6c9ab46b34b8   6 weeks ago          237MB
opsflow-app                     latest    8d3348b15159   6 months ago         2.05GB
```

Step 3: View Image Details

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Continerization And DevOps Lab\experiment-4> docker build -t saharsh-kumar:1.0 .
[+] Building 34.9s (11/11) FINISHED
 => [internal] load build defintion from Dockerfile
 => => transferring dockerfile: 381B
 => [internal] load metadata for docker.io/library/python:3.9-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 304B
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aablacdale13cf1731b1b
 => => sha256:ea56f688404adf81680322f152d2cfec62115b30dd4a881c2c450078315beb508   251B / 251B
 => => sha256:fc74430849022d13b0d44bb8969a953f842f59c6e9d1a0c2c83d710af18286c08   13.88MB / 13.88MB
 => => sha256:b3ec39b36ae8c03a3ee09854de4ec4aa08381dfed84a9daa0075048c2e3df8881d   1.29MB / 1.29MB
 => => sha256:38513bd72256313495cddd83bb0915a33cfa475dc2a07072ab2c8d191020ca5d   29.78MB / 29.78MB
 => => extracting sha256:38513bd72256313495cddd83bb0915a33cfa475dc2a07072ab2c8d191020ca5d
 => => extracting sha256:b3ec39b36ae8c03a3ee09854de4ec4aa08381dfed84a9daa0075048c2e3df8881d
 => => extracting sha256:fc74430849022d13b0d44bb8969a953f842f59c6e9d1a0c2c83d710af18286c08
 => => extracting sha256:ea56f688404adf81680322f152d2cfec62115b30dd4a81c2c450078315beb508
 => [internal] load build context
 => => transferring context: 335B
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:72ea78de2919391cd164f5efffbff2fd90020d0a:ab72a6892ad3cfa3bff640336
 => => exporting config sha256:813a61daa693e5106967ae2bef9900d1f4d8faab442eb88f8888e82dd789a6c3f8d
 => => exporting attestation manifest sha256:a132f73af3247950dbdae7b9f6be1123fc300d7e56515b9c10aa3f3f0
 => => exporting manifest list sha256:0cb845d28b044b208f43e05c2e71082e09957c8974c2822102653553d9eda07
 => => naming to docker.io/library/saharsh-kumar:1.0
 => => unpacking to docker.io/library/saharsh-kumar:1.0
```

Part 4: Running Containers

Step 1: Run Container

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4> # List all images
>> docker images
>>
>> # Show image history
>> docker history my-flask-app
>>
>> # Inspect image details
>> docker inspect my-flask-app
>>
REPOSITORY                        TAG       IMAGE ID       CREATED          SIZE
my-flask-app                      1.0       3179d3c3cc8f   4 minutes ago    200MB
my-flask-app                      latest    3179d3c3cc8f   4 minutes ago    200MB
my-flask-app                      v1.0      3179d3c3cc8f   4 minutes ago    200MB
username/my-flask-app             1.0       971699264f9c   4 minutes ago    200MB
web_app_project-backend           latest    a1b113fcb6cd   2 days ago       187MB
containerization_web_app-backend  latest    eabf88c5f99a   3 days ago       187MB
web_app_project-db                latest    7ec9d7a46981   3 days ago       392MB
containerization_web_app-db       latest    921d26fa4265   3 days ago       392MB
nginx-alpine                      latest    85b49b9478d8   3 weeks ago      16MB
nginx-ubuntu                      latest    db766c1efb20   3 weeks ago      199MB
nginx                             latest    341bf0f3ce6c   6 weeks ago      237MB
<none>                            <none>    6c9ab46b34b8   6 weeks ago      237MB
opsflow-app                       latest    8d3348b15159   6 months ago     2.05GB

IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
3179d3c3cc8f   4 minutes ago    CMD ["python" "app.py"]                         0B        buildkit.dockerfile.v0
<missing>      4 minutes ago    EXPOSE 0.0.0.0:5000                             0B        buildkit.dockerfile.v0
<missing>      4 minutes ago    COPY app.py . # buildkit                        12.3kB    buildkit.dockerfile.v0
<missing>      4 minutes ago    RUN /bin/sh -c pip install --no-cache-dir                 buildkit.dockerfile.v0
<missing>      4 minutes ago    requirements.txt # buildkit                     13.4MB    buildkit.dockerfile.v0
<missing>      4 minutes ago    COPY requirements.txt .  # buildkit             12.3kB    buildkit.dockerfile.v0
<missing>      4 months ago     WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      4 months ago     CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   for src in idle3 p…  16.4kB    buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   savedAptMark='$((a…  45.7MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=000e074/c0f2f0ce082432d1ee8…  0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25                       0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E13…  0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux;   apt-get update;   a…  4.94MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@l…   87.4MB    debuerrreotype 0.16
 [{
       "Id": "sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f8dda002a4c21724",
       "RepoTags": [
           "my-flask-app:1.0",
           "my-flask-app:latest",
           "my-flask-app:v1.0"
       ],
       "RepoDigests": [
           "my-flask-app@sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f8dda002a4c21724"
       ],
       "Parent": "",
       "Comment": "buildkit.dockerfile.v0"
 }]
```

Step 2: Manage Containers

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem/Containerization_And_DevOps_Lab\experiment-4> # Run container with port mapping
>> docker run -d -p 5000:5000 -name saharsh-kumar-flask-app
e3f6856178581e54904746fc0d6b72cc1b6fdba26406dd006b45ef0e3722350b

>> # Visit the application
>> http://localhost:5000 _

>> # View container logs
>> docker logs saharsh-kumar-flask-app

docker: Error response from daemon: Conflict. The container name "/saharsh-kumar-flask-app" is already in use by container "e3f6856178581e54904746fc0d6b72cc1b6fdba26406dd006b45eff
[illegible] (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
    RECOMMENDED ACTION:
    Use the -UseBasicParsing switch to avoid script code execution.
     Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A

StatusCode        : 200
StatusDescription : OK
Content           : Hello from Docker!
RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 18
                    Content-Type: text/html; charset=utf-8
                    Date: Thu, 19 Mar 2026 16:29:23 GMT
                    Server: Werkzeug/3.1.6 Python/3.9.25

                    Hello from Docker!
Forms             : {}
Headers           : {[Connection, close], [Content-Length, 18], [Content-Type, text/html; charset=utf-8], [Date, Thu, 19 Mar 2026 16:29:23...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 18

CONTAINER ID   IMAGE                     COMMAND           CREATED              STATUS              PORTS                                       NAMES
e3f685617858   saharsh-kumar-flask-app   "python app.py"   About a minute ago   Up About a minute   0.0.0.0:5000->5000/tcp, [::]:5000>          saharsh-kumar-flask-app

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
127.0.0.1 - - [19/Mar/2026 16:29:23] "GET / HTTP/1.1" 200 -
```

Part 5: Multi-stage Builds

Step 1: Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step 2: Simple Multi-stage Dockerfile

Dockerfile.multistage

```dockerfile
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

Step 3: Build and Compare

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4> # Stop container
>> docker stop saharsh-kumar-flask-container
>>
>> # Start stopped container
>> docker start saharsh-kumar-flask-container
>>
>> # Remove container
>> docker rm saharsh-kumar-flask-container
>>
>> # Remove container forcefully
>> docker rm -f saharsh-kumar-flask-container
>>
saharsh-kumar-flask-container
saharsh-kumar-flask-container
Error response from daemon: cannot remove container "saharsh-kumar-flask-container": container is running: stop the container
before removing or force remove
saharsh-kumar-flask-container
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4>  docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qvka4pbwiexffg6hxjgsiuryf
flask-multistage                latest    e17ac3145f61   1 second ago     219MB
username/my-flask-app           1.0       971699264f9c   21 minutes ago   200MB
my-flask-app                    1.0       3179d3c3cc8f   21 minutes ago   200MB
my-flask-app                    latest    3179d3c3cc8f   21 minutes ago   200MB
my-flask-app                    v1.0      3179d3c3cc8f   21 minutes ago   200MB
flask-regular                   latest    518e70674bd4   21 minutes ago   200MB
```

Part 6: Publishing to Docker Hub

Step 1: Prepare for Publishing

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Contaninerization_And_DevOps_Lab\experiment-4> # Login to
>> docker login
>>
>> # Tag image for Docker Hub
>> docker tag my-flask-app:latest  saharsh-kumar/my-flask-app:1.0
>> docker tag my-flask-app:latest  saharsh-kumar/my-flask-app:latest
>>
>> # Push to Docker Hub
>> docker push  saharsh-kumar/my-flask-app:1.0
>> docker push  saharsh-kumar/my-flask-app:latest
>>
Authenticating with existing credentials... [Username: saharsh-kumar]
i Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
The push refers to repository [docker.io/saharsh-kumar/my-flask-app]
8cf145093bb8: Pushed
65931fd06eb6: Pushed
fc7443084902: Pushed
ea56f685404a: Pushed
b3ec39b36ae8: Pushed
38513bd72563: Pushed
fd0f088c4a1d: Pushed
2b3939b7a465: Pushed
62898ac0e499: Pushed
1.0: digest: sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f8dda002a4c21724 size: 856
The push refers to repository [docker.io/saharsh-kumar/my-flask-app]
62898ac0e499: Layer already exists
ea56f685404a: Layer already exists
65931fd06eb6: Layer already exists
fd0f088c4a1d: Layer already exists
fc7443084902: Layer already exists
8cf145093bb8: Layer already exists
38513bd72563: Layer already exists
b3ec39b36ae8: Layer already exists
2b3939b7a465: Layer already exists
latest: digest: sha256:3179d3c3cc8f8000d22418c5c6c7241f6f8dc7f4dc8ea497f8dda002a4c21724 size: 856
```

Step 2: Pull and Run from Docker Hub

Pull from Docker Hub (on another machine)

docker pull username/my-flask-app:latest

Run the pulled image

docker run -d -p 5000:5000 username/my-flask-app:latest

Part 7: Node.js Example (Quick Version)

Step 1: Node.js Application

```
PS C:\Users\aharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4> mkdir my-node-app
>> cd my-node-app
>>


    Directory: C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4\my-node-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        19-03-2026     23:02                my-node-app
```

app.js

```javascript
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
    res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
    res.json({ status: 'healthy' });
});

app.listen(port, () => {
    console.log(`Server running on port ${port}`);
});
```

package.json

```json
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step 2: Node.js Dockerfile

```dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step 3: Build and Run

```
PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4> mkdir my-node-app
>> cd my-node-app
>>

    Directory: C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\Containerization_And_DevOps_Lab\experiment-4\my-node-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        19-03-2026     23:02                my-node-app

PS C:\Users\saharsh kumar\OneDrive\Desktop\UPES\6th Sem\ContainerizationAnd_DevOps_Lab\lexperiment-4\my-node-app> # Build image
>> docker build -t my-node-app .
>>
>> # Run container
>> docker run -d -p 3000:3000 --name node-container my-node-app
>>
>> # Test
>> curl http://localhost:3000
[+] Building 5.3s (11/11) FINISHED
 => [internal] load build definition from Dockerfile                                                        0.0s
 => => transferring dockerfile: 194B                                                                        0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                           1.4s
 => [auth] library/node:pull token for registry-1.docker.io                                                 0.0s
 => [internal] load .dockerignore                                                                           0.0s
 => => transferring context: 28B                                                                            0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.0s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e   0.0s
 => [internal] load build context                                                                           0.0s
 => => transferring context: 539B                                                                           0.0s
 => CACHED [2/5] WORKDIR /app                                                                               0.0s
 => [3/5] COPY package*.json ./                                                                             0.0s
 => [4/5] RUN npm install --only=production                                                                 2.3s
 => [5/5] COPY app.js .                                                                                     0.0s
 => exporting to image                                                                                      0.1s
 => => exporting layers                                                                                     0.1s
 => => exporting manifest sha256:360adb219fc4108dc17546526a5cb8f50a8e89230d31ca809ef29c299d42169            0.0s
 => => exporting config sha256:8be84c993a86fff0eb0e93703d9ad6ea3633fff9b9557885788070db1b37f219771          0.0s
 => => exporting attestation manifest sha256:5216855fdbde5218a52553bec4079bff36c002ea00984ffd8dead45747c9d43   0.0s
 => => exporting manifest list sha256:db3e380a66a0fa2b8e4f40fa483549cc68838b6e16499cd3d04d162528dddff09b3   0.0s
 => => naming to docker.io/library/my-node-app:latest                                                       0.0s
 => => unpacking to docker.io/library/my-node-app:latest                                                    0.0s
```

Part 8: Quick Practice Exercises

Exercise 1: Tagging Practice

```
# Create an image with three tags:
# 1. myapp:latest
# 2. myapp:v2.0
# 3. yourusername/myapp:production

# Solution:
docker build -t myapp:latest -t myapp:v2.0 -t username/myapp:production .
```

Exercise 2: Multi-stage for Node.js

```
# Create a multi-stage Dockerfile for Node.js that:
# 1. Uses builder stage for npm install
# 2. Creates final image with only production dependencies
# 3. Uses non-root user

# Hint:
# STAGE 1: FROM node:18-alpine AS builder
# STAGE 2: FROM node:18-alpine
# COPY --from=builder /app/node_modules ./node_modules
```

Exercise 3: Clean Build

```
# Build without cache and with .dockerignore
docker build --no-cache -t clean-app .

# Compare with cached build
time docker build -t cached-app .
```

Essential Docker Commands Cheatsheet

Command	Purpose	Example
docker build	Build image	docker build -t myapp .

Command	Purpose	Example
docker run	Run container	docker run -p 3000:3000 myapp
docker ps	List containers	docker ps -a
docker images	List images	docker images
docker tag	Tag image	docker tag myapp:latest myapp:v1
docker login	Login to Dockerhub	`echo "token"
docker push	Push to registry	docker push username/myapp
docker pull	Pull from registry	docker pull username/myapp
docker rm	Remove container	docker rm container-name
docker rmi	Remove image	docker rmi image-name
docker logs	View logs	docker logs container-name
docker exec	Execute command	docker exec -it container-name bash

---

Common Workflow Summary

Development Workflow:

1. Create Dockerfile and .dockerignore

2. Build image

docker build -t myapp .

3. Test locally

docker run -p 8080:8080 myapp

4. Tag for production

docker tag myapp:latest myapp:v1.0

5. Push to registry

docker push myapp:v1.0

Production Workflow:

1. Pull from registry

docker pull myapp:v1.0

2. Run in production

docker run -d -p 80:8080 --name prod-app myapp:v1.0

3. Monitor

docker logs -f prod-app

Key Takeaways

1. **Dockerfile** defines how to build your image.
2. **.dockerignore** excludes unnecessary files (important for security and performance).
3. **Tagging** helps version and organize images.
4. **Multi-stage builds** create smaller, more secure production images.
5. **Docker Hub** allows sharing and distributing images.
6. Always test images locally before publishing.

Cleanup

Remove all stopped containers

docker container prune

Remove unused images

docker image prune

Remove everything unused

docker system prune -a